

Ozygen University

Computer Science, Faculty of Engineering

Parallel Computing

Project №1

Student: Mokrov Semen

CS 419.A

Teacher: Nitel Muhtaroglu

Istanbul, 2023

Table of contents

1. Introduction

- a. Problem overview
- b. Technology stack

2. The similarity check algorithm

- a. Jaccard index
- b. Analogs

3. Preparing the input content

- a. Normalization
- b. Supported formats

4. Serial algorithm

5. Parallel algorithm

6. Hardware

7. Testing

- a. Graph
- b. Description
- c. Supercomputer testing – examples, datasets, test cases
- d. Graphs and results

8. What may be improved?

9. References

Introduction

Problem overview

Finding similarities between datasets involves analyzing the degree of resemblance or correspondence between different sets of data. It is a crucial task in various fields, including data integration, data cleaning, data matching, machine learning, and data exploration. The goal is to identify common patterns, relationships, or entities within the datasets to derive meaningful insights and enable effective data management.

The process of finding similarity typically involves comparing data elements or instances and quantifying their resemblance. Several approaches and techniques can be employed depending on the nature of the data and the specific requirements of the analysis.

This report describes one of the implementations of the finding the similarity.

Technology stack

In this project the main programming language was C, specification is C99. As an IDE CLion was used also, the program runs in the CLion virtual environment constructed by the CMake utility.

For communication between processes MPI library (<mpi.h>) was used, the program was prepared as .exe file (via CMake) and was run by the command:

```
mpiexec -n <amount of processes> ./cmake-build-release/parallel_computing_project.exe
```

For the business-logic was used only C-native utilities.

As a VCS, GitLab was used.

The similarity check algorithm

Jaccard index

Widely used in various fields, including data mining, information retrieval, and pattern recognition. The Jaccard Index quantifies the overlap or agreement between two sets by calculating the ratio of the size of their intersection to the size of their union.

To understand how the Jaccard Index helps in finding similarity, let's consider two sets, A and B. The Jaccard Index (J) is calculated using the following formula:

$$J(A, B) = |A \cap B| / |A \cup B|$$

In this formula, $|A \cap B|$ represents the size of the intersection of sets A and B, while $|A \cup B|$ represents the size of their union.

The resulting Jaccard Index ranges from 0 to 1. A Jaccard Index of 0 indicates that the two sets have no common elements, implying maximum dissimilarity. Conversely, a Jaccard Index of 1 indicates that the two sets are identical, indicating maximum similarity.

In the current project, this method was used to detect

Analogs

There are several more methods for detecting similarity between datasets, each suitable for different types of data and analysis objectives. Here are several commonly used methods:

1. Distance Metrics: Distance metrics quantify the dissimilarity between data points or instances. Commonly used metrics include Euclidean distance, Manhattan distance, cosine similarity, and Hamming distance. These metrics measure the numerical or structural differences between data points, allowing for similarity comparison.

2. **Correlation Analysis:** Correlation analysis measures the statistical relationship between variables in datasets. Techniques such as Pearson correlation coefficient or Spearman's rank correlation coefficient assess the strength and direction of the linear or monotonic relationship between variables. High correlation values indicate similarity or dependency between datasets.
3. **Clustering:** Clustering algorithms group similar data points together based on their intrinsic properties. By clustering datasets, similarities between groups or clusters can be identified. Common clustering algorithms include k-means, hierarchical clustering, and density-based clustering. Clustering helps in identifying natural structures or patterns within datasets.

Preparing the input content

Normalization

Normalizing data is the process of transforming data into a standardized scale or range to eliminate variations in the data's magnitude. Normalization is crucial in data analysis, as it ensures fair comparisons and prevents certain variables from dominating the analysis due to their larger scales. Here are several common ways to normalize data:

1. Min-max scaling, also known as normalization, rescales the data to a specific range, typically between 0 and 1. The formula to perform min-max scaling is:
2. $X_{\text{normalized}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$. Where $X_{\text{normalized}}$ represents the normalized value, X is the original value, X_{min} is the minimum value in the dataset, and X_{max} is the maximum value in the dataset.
3. Z-score standardization transforms data to have a mean of 0 and a standard deviation of 1. This method is useful when the distribution of the data is approximately Gaussian (bell-shaped). The formula to calculate the z-score is: $Z = (X - \mu) / \sigma$. Where Z represents the standardized value, X is the original value, μ is the mean of the dataset, and σ is the standard deviation of the dataset.
4. Decimal scaling involves shifting the decimal point of the values to achieve normalization. The decimal point is moved to the left or right based on the maximum absolute value in the dataset. The number of decimal places is determined by the number of digits required to represent the maximum absolute value.

The project requires multiplying each element in the dataset before running the program on 100 to make it between [0,100] values.

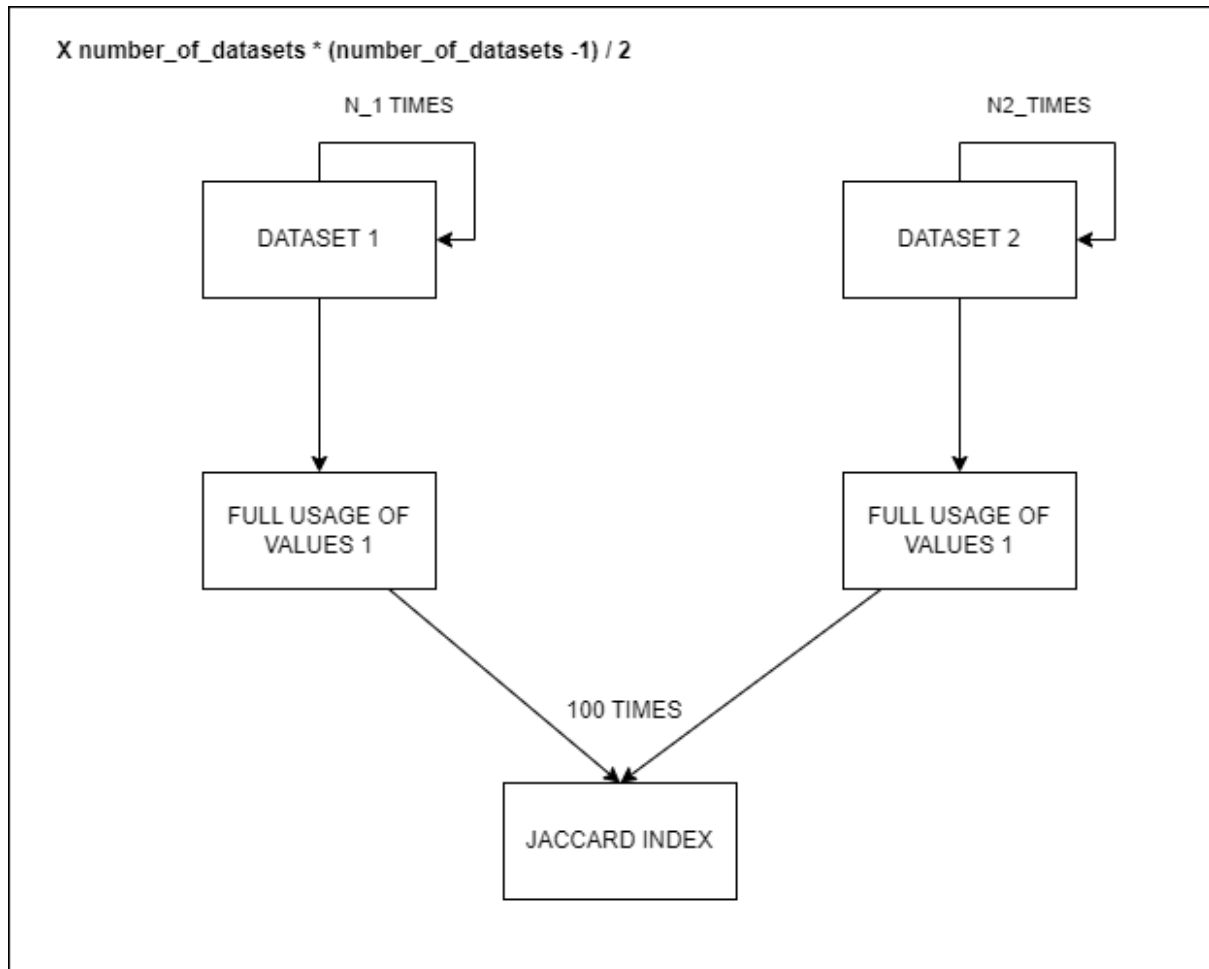
Supported formats

The program requires writing the number of datasets, the pathname for each one, and the number of values in each one.

All extension that lets write text in UTF-8 is supported, datasets must be in files (values must be one by one, divided by comma)

Serial algorithm

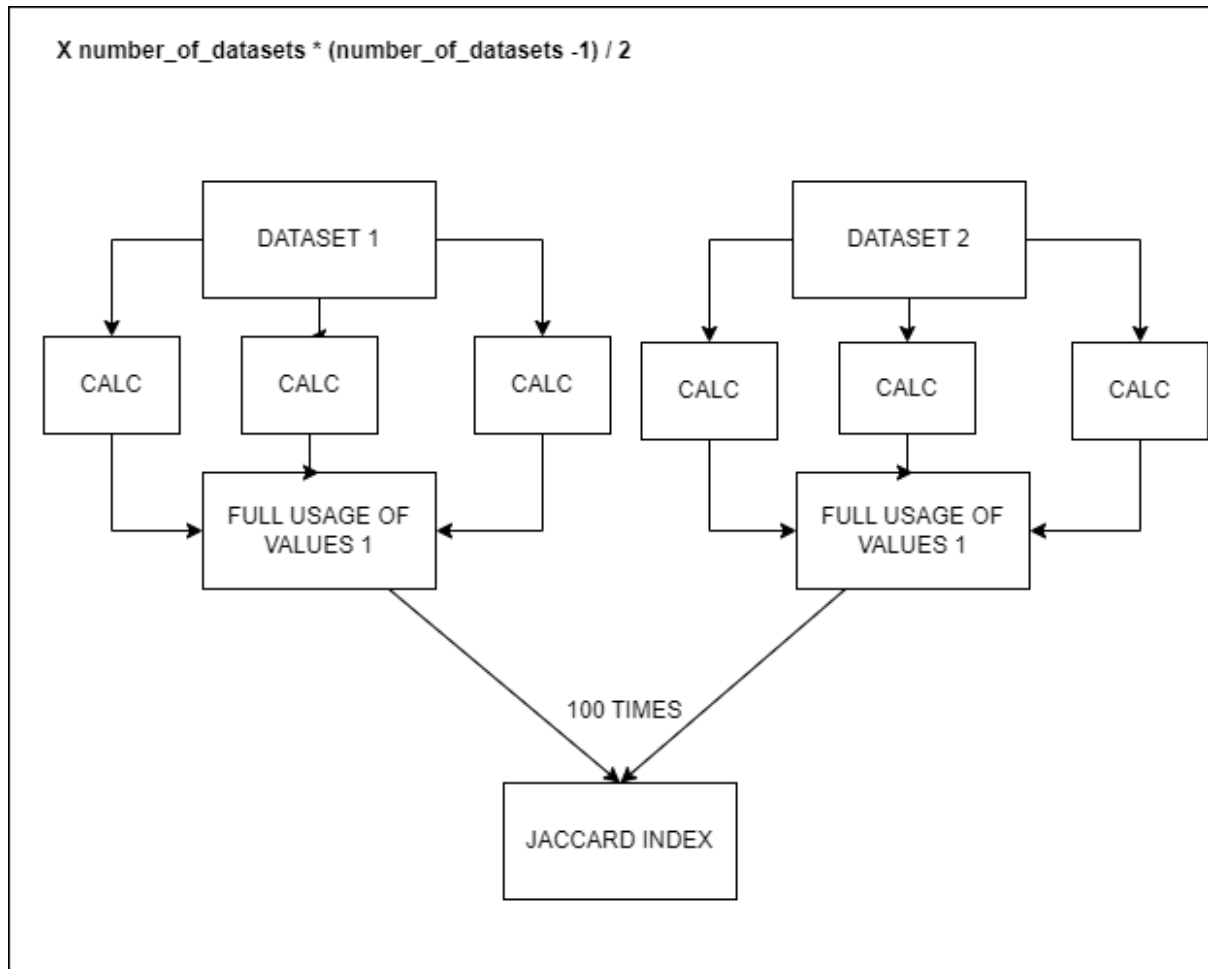
The main idea of the sequential calculation of the Jaccard index for each pair of presented datasets is to calculate the number of times that each character (from 0 to 100) occurs in each dataset, and later, by comparing the results obtained, directly calculate this index. Below is a diagram of the algorithm that is processing each dataset:



We go through each element of the dataset, remembering how often it was selected, after which we compare the two results obtained in constant time. Since we have N datasets, we will have to do it $N * (N - 1) / 2$ times.

Parallel algorithm

Since the number of datasets (even squared) based on the project conditions is disproportionately small, I decided to parallelize the process of counting the number of elements encountered in each dataset (since the external cycle is already much smaller than the internal one, I did not see the need for parallelizing each stage of calculating the coefficient for 2 datasets).



To exchange data and divide the task area of each process, MPI functions were used: MPI_Scatter (scatter, distributes data from one process to others, distributing them evenly), MPI_Gather (gather, collects data from all processes into one) and MPI_Bcast (broadcast, transmits complete data from one process to all others).

The benefit of parallelization depends on the maximum number of elements in one of the datasets and on the number of processes.

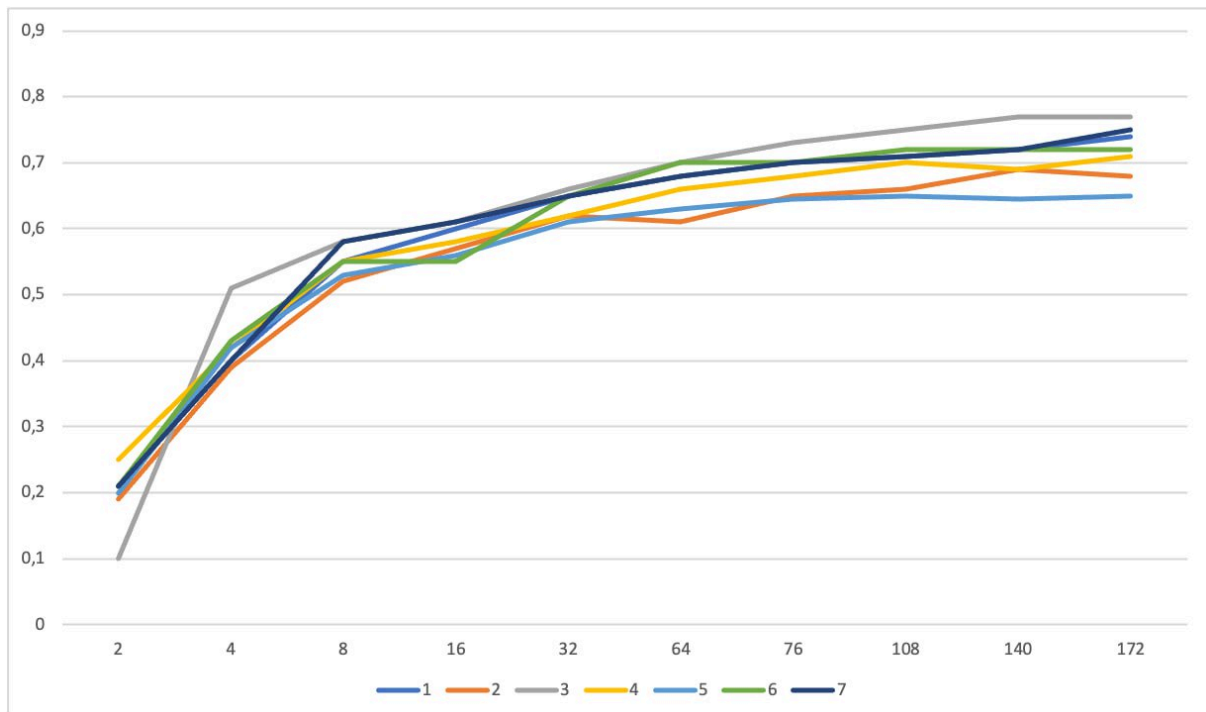
Hardware

The characteristics of the Xiaomi Mi Pro 15.6:

Characteristic	Description
Display	15.6-inch IPS LCD display
Resolution	1920 × 1080 pixels
Processor	Intel Core i7-1165G7 11th Gen quad-core processor
Graphics	NVIDIA GeForce MX450 GPU with 2GB GDDR6 VRAM
Memory	16GB DDR4 RAM
Storage	512GB PCIe NVMe SSD
Operating System	Windows 10 Home
Connectivity	Wi-Fi 6, Bluetooth 5.1, USB Type-C, HDMI, USB-A, 3.5mm audio
Ports	1 x USB-C, 1 x USB-A 3.2 Gen 1, 2 x USB-A 2.0, 1 x HDMI, 1 × 3.5mm audio jack
Battery	66Wh battery with fast charging support
Keyboard	Backlit keyboard with a numeric keypad
Webcam	Integrated 720p HD webcam
Audio	Harman/Kardon speakers, DTS Audio processing
Weight	Approximately 1.8 kg (4.0 lbs)
Dimensions	348.4mm x 237.5mm x 16.3mm (13.72" x 9.35" x 0.64")

Testing

Graph



Description

Testing was carried out on small datasets, so there is no significant difference, but you can still notice the main trends:

- The larger the dataset size, the slower the processing time grows with more processes
- If there is not enough data, increasing the number of processors only hinders since the overhead exceeds the profit
- On a small data set, the difference is most striking, it decreases over time

Supercomputer testing – examples, datasets, test cases

Examples

There are several examples for testing the same as for typical work of project, but with much more numbers: maximum is 10 millions number, minimum is 1 million numbers. Datasets were created randomly, the fact that the answer of programs is correctly approved by the serial program.

Test cases

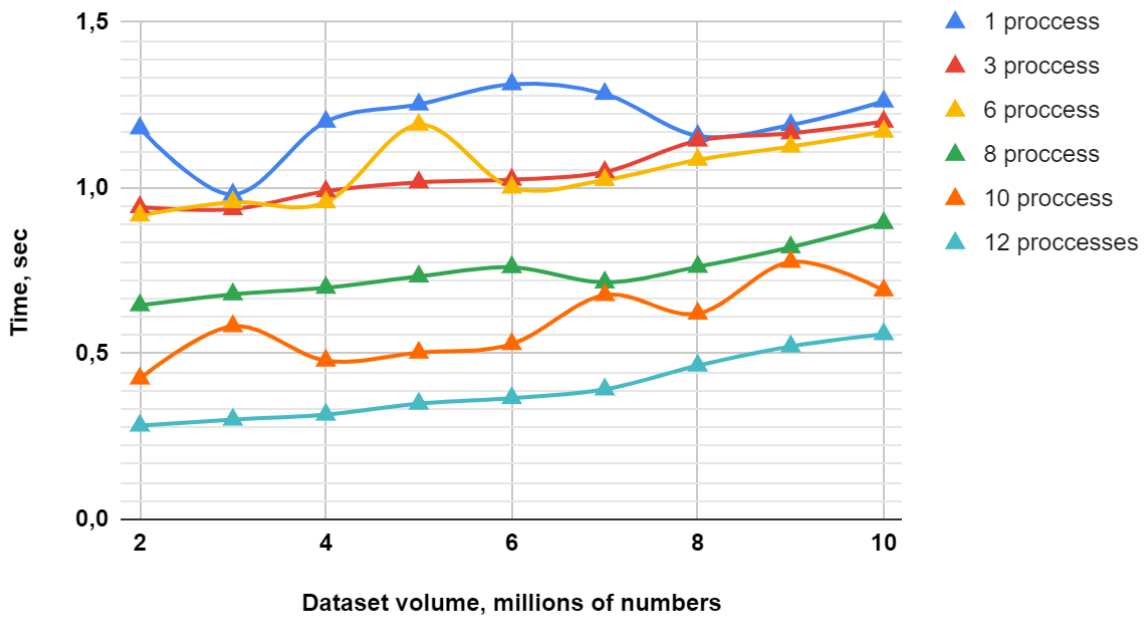
For each chosen amount of processes: 1, 3, 6, 8, 10, 12 (were chosen by experiment) there are test cases for all similar datasets with the amount of numbers between one and ten million. Also, there is a test case with shuffling different size datasets between themselves for a test algorithm on distributivity.

Graphs and results

On the first test with all equal datasets for every amount of processes I got (axis X – numbers in datasets, axis Y – time in ms (sec on the graph), on the top – amount of proccesses):

	1	3	6	8	10	12
1	1190	905	887	575	392	263
2	1180	942	917	646	425	283
3	980	936	957	679	583	301
4	1200	990	957	699	479	316
5	1252	1017	1191	733	503	349
6	1313	1025	1001	761	529	366
7	1284	1048	1024	715	677	392
8	1157	1143	1085	762	621	464
9	1190	1165	1125	821	777	522
10	1261	1200	1170	894	691	559

Comparison on datasets with equal size

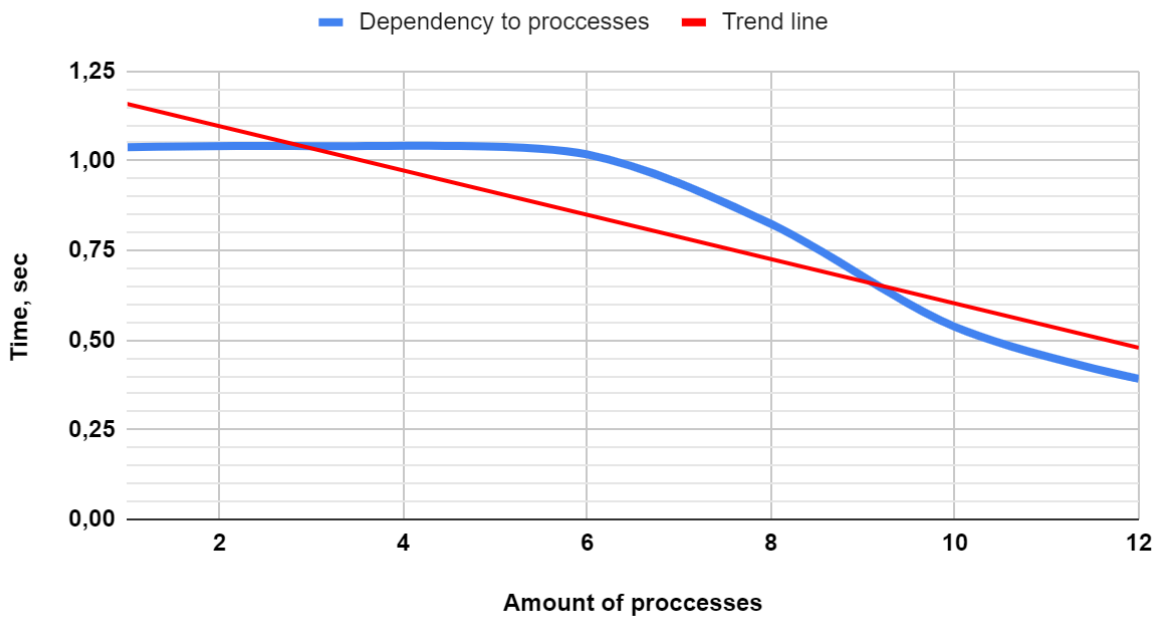


We're able to see that with increasing number of processes effectiveness of the calculation is also increasing. Certainly, there are some errors and emissions, but it's understandable and doesn't break common rule.

For test with 10 dataset with different size I go such result (left – amount of processes, right – elapsed time):

12	392
10	537
8	824
6	1018
3	1041
1	1038

Behaviour with different size datasets (10 datasets)



We also can see that the trend line tilted down and tends to zero. That means that up to this stage with increasing amount of processes we're also increasing effectiveness.

Result of testing on supercomputer:

Amount of processes: 1
Amount of values: 1000000
7932
Start: 0, Finish: 1 -- Coeff: 0.986956
Elapsed time (ms): 1261

Amount of values: 900000
2770
Start: 0, Finish: 1 -- Coeff: 0.988607
Elapsed time (ms): 1190

Amount of values: 800000
3932
Start: 0, Finish: 1 -- Coeff: 0.987755
Elapsed time (ms): 1157

Amount of values: 700000
3917
Start: 0, Finish: 1 -- Coeff: 0.987185

Elapsed time (ms): 1284

Amount of values: 600000
5749
Start: 0, Finish: 1 -- Coeff: 0.984796
Elapsed time (ms): 1313

Amount of values: 500000
4465
Start: 0, Finish: 1 -- Coeff: 0.985537
Elapsed time (ms): 1252

Amount of values: 400000
3364
Start: 0, Finish: 1 -- Coeff: 0.985176
Elapsed time (ms): 1200

Amount of values: 300000

4530
Start: 0, Finish: 1 -- Coeff: 0.975556
Elapsed time (ms): 980

Amount of values: 200000
5648

Start: 0, Finish: 1 -- Coeff: 0.976646
Elapsed time (ms): 1180

Amount of values: 100000
9376

Start: 0, Finish: 1 -- Coeff: 0.968291
Elapsed time (ms): 1190

79653065541732173039

Start: 0, Finish: 1 -- Coeff: 0.964000
Start: 0, Finish: 2 -- Coeff: 0.966278
Start: 0, Finish: 3 -- Coeff: 0.964000
Start: 0, Finish: 4 -- Coeff: 0.964849
Start: 0, Finish: 5 -- Coeff: 0.962554
Start: 0, Finish: 6 -- Coeff: 0.966162
Start: 0, Finish: 7 -- Coeff: 0.960323
Start: 0, Finish: 8 -- Coeff: 0.966278
Start: 0, Finish: 9 -- Coeff: 0.965177
Start: 1, Finish: 2 -- Coeff: 0.968872
Start: 1, Finish: 3 -- Coeff: 1.000000
Start: 1, Finish: 4 -- Coeff: 0.968872
Start: 1, Finish: 5 -- Coeff: 0.965525
Start: 1, Finish: 6 -- Coeff: 0.966356
Start: 1, Finish: 7 -- Coeff: 0.962150

Start: 1, Finish: 8 -- Coeff: 0.968872
Start: 1, Finish: 9 -- Coeff: 0.966046
Start: 2, Finish: 3 -- Coeff: 0.968872
Start: 2, Finish: 4 -- Coeff: 0.970754
Start: 2, Finish: 5 -- Coeff: 0.967691
Start: 2, Finish: 6 -- Coeff: 0.966916
Start: 2, Finish: 7 -- Coeff: 0.968000
Start: 2, Finish: 8 -- Coeff: 1.000000
Start: 2, Finish: 9 -- Coeff: 0.967265
Start: 3, Finish: 4 -- Coeff: 0.968872
Start: 3, Finish: 5 -- Coeff: 0.965525
Start: 3, Finish: 6 -- Coeff: 0.966356
Start: 3, Finish: 7 -- Coeff: 0.962150
Start: 3, Finish: 8 -- Coeff: 0.968872
Start: 3, Finish: 9 -- Coeff: 0.966046
Start: 4, Finish: 5 -- Coeff: 0.967497
Start: 4, Finish: 6 -- Coeff: 0.966549
Start: 4, Finish: 7 -- Coeff: 0.966742
Start: 4, Finish: 8 -- Coeff: 0.970754
Start: 4, Finish: 9 -- Coeff: 0.966510
Start: 5, Finish: 6 -- Coeff: 0.964579
Start: 5, Finish: 7 -- Coeff: 0.966008
Start: 5, Finish: 8 -- Coeff: 0.967691
Start: 5, Finish: 9 -- Coeff: 0.964154
Start: 6, Finish: 7 -- Coeff: 0.966317
Start: 6, Finish: 8 -- Coeff: 0.966916
Start: 6, Finish: 9 -- Coeff: 0.964907
Start: 7, Finish: 8 -- Coeff: 0.968000
Start: 7, Finish: 9 -- Coeff: 0.965699
Start: 8, Finish: 9 -- Coeff: 0.967265
Elapsed time (ms): 1038

What may be improved?

1. The process of matching the results for each of the pairs can also be parallelized -- each free process is allocated 2 pairs, and it returns an index (hard to implement)
2. Depending on the number of processes, you can build a topological network (for example, a hypercube or a ring) to improve the speed of interaction between processes
3. You can limit the number of processes to an even number with the maximum size of the dataset -- by type of implementation and MPI specification, scatter requires transferring to each process a volume of data of the same size, so under some conditions some processes are idle.

References

1. <https://www.analyticsvidhya.com/blog/2021/02/a-simple-guide-to-metrics-for-calculating-string-similarity/>
2. <https://lms.ozyegin.edu.tr/course/view.php?id=4291>
3. <https://www.mpi-forum.org/docs/>
4. <https://www.mpich.org/static/docs/latest/>
5. <https://curc.readthedocs.io/en/latest/programming/MPI-C.html>
6. Repo link: https://gitlab.com/semen.mokrov.ozu/parallel_computing/